

A STEP-BY-STEP TUTORIAL WITH CODE AND OUTPUT

feynsage

*Feynman integrals in SageMath,
the lecture in code and a real prediction*

Kazi Abu Rousan

School of Physical Sciences, NISER, Bhubaneswar

Source code and install instructions at github.com/aburousan/feynsage

WHAT THIS TUTORIAL IS

It teaches the package feynsage by doing with it the calculations of Prof. B. Ananthanarayan's two-part lecture "An Introduction to Feynman Integrals" (NISER, 2026). These are the tadpole, integration by parts, the bubble and its differential equation, Feynman parameters and the graph polynomials $\mathcal{U}$ and $\mathcal{F}$. Each function is explained in a violet box the first time it is used. The box says what the function takes, what it gives back and what it saves you from doing by hand. The last chapter puts the pieces together for a real prediction (the lifetime of the neutral pion) and compares it with experiment.

Every grey box marked **In** [n] is real code. It was run, in this order, in one SageMath session (as in a Jupyter notebook) and the box under it marked **Out** [n] is what came out. Nothing in the output boxes was typed by hand. I am also writing up the lecture itself in full (my notes *Feynman integrals at one loop*). They are not finished yet and will be shared once they are complete.

I wrote feynsage in 2024 for my MSc project. After sir's talk it was refined, checked against the calculations of the lecture and published at github.com/aburousan/feynsage under the MIT licence.

Contents

1	Getting feynsage	3
1.1	What you need	3
1.2	Download and install	3
1.3	Check and start	3
2	A short tour of feynsage	3
3	Part 1: the tadpole	4
3.1	The solid angle	4
3.2	The tadpole with n powers	4
3.3	The pole and the $\lambda\varphi^4$ tadpole	5
4	Part 1: integration by parts for the tadpole	6
4.1	Integral families	6
4.2	The IBP identity	6
4.3	Solving the identities: reduction	7
4.4	The difference equation and its solution	8
5	Part 1: the basis of one-loop integrals	8
6	Part 1: the bubble	9
6.1	IBP for the bubble	9
6.2	The differential equation	10
6.3	Solving the differential equation	11
6.4	The hypergeometric function in Sage	13
7	Part 1: the fish diagram and Feynman parameters	13
7.1	Ramond's expansion around $D = 4$	14
7.2	Going to Minkowski space: the imaginary part	14
8	Part 2: graph polynomials	15
8.1	The bubble with two masses	16
8.2	The box (Smirnov, Fig. 3.6)	16
8.3	Smirnov's two-loop example	17
8.4	From $\mathcal{U}$ and $\mathcal{F}$ back to the integral	18
8.5	Sir's notebook: completing the square	19
9	A real prediction: the lifetime of the neutral pion	20
9.1	Step 1: the graph, its trees and its polynomials	20
9.2	Step 2: the integral from $\mathcal{U}$ and $\mathcal{F}$	22
9.3	Step 3: the Dirac trace	23
9.4	Step 4: the amplitude and why the quark mass drops out	24
9.5	Step 5: the decay rate	24
10	Where to go next	26

1 Getting feynsage

1.1 What you need

- **SageMath** 10.7 or newer. It is free. The install script below finds it or installs it.
- **FORM** for the Dirac traces of the last chapter. The script installs it too.
- macOS (Apple silicon or Intel), Linux, or Windows through WSL. SageMath does not run on Windows itself.

1.2 Download and install

The code is at github.com/aburousan/feynsage. Either clone it with git or, on the web page, press **Code** and then **Download ZIP** and unpack it. Then, inside the folder:

```
git clone https://github.com/aburousan/feynsage.git
cd feynsage
./install.sh          # finds or installs SageMath and FORM, then installs feynsage
./install.sh --test   # the same, then runs the tests
```

If SageMath is already there, `sage -pip install --user --no-deps .` does the last step by hand.

1.3 Check and start

```
sage -c "import feynsage; print(feynsage.__file__)" # where it was installed
sage -n jupyterlab                               # notebooks, choose the SageMath kernel
```

Every cell of this tutorial can be run again with `sage docs/tutorial/run_cells.sage`.

2 A short tour of feynsage

feynsage is a Python package that runs inside SageMath, so all of Sage (symbols, exact fractions, calculus, plots) is available next to it. Load it once:

```
In [1]: from feynsage import *
        from feynsage.one_loop import tadpole, bubble_equal_mass, expand_eps, D
```

The first line loads everything that is used most. The second loads a few closed formulas from the module `feynsage.one_loop`. The functions used in this tutorial fall into five groups:

task	functions	lecture
closed formulas	tadpole, bubble_equal_mass, bubble_massless, expand_eps	tadpole, bubble, the ε expansion
families and IBP	family, .ibp, ibp_reduce, .draw, .UF	IBP identities, reduction, completing the square
one-loop functions	A0, B0, C0, D0, finite_part, quick_plot	the basis A, B, C, D
graphs	graph, .U, .F, .spanning_trees, .two_forests, .plot, draw_panels	1-trees, 2-trees, $\mathcal{U}$, $\mathcal{F}$
Dirac traces	form.dirac_trace, form.compute (traces and contractions in Sage notation, converted to FORM)	the pion (last chapter)

These are only the functions this tutorial needs. feynsage has many more that the lecture did not reach. Some of them are one-loop tensor integrals in the style of Package-X (loop, PVB, PVC, PVD), closed forms of C_0 and D_0 in dilogarithms (explicit, c0_closed, d0_closed), infrared poles (the module ir), the finite-field reducer reduce_ff, symmetries, diagram (ready-made graphs like "kite") and pictures of sectors (draw_sectors). All of them are listed with their arguments in [docs/REFERENCE.md](#) in the repository and the notebook [examples/feynsage_walkthrough.ipynb](#) uses most of them. `info(f)` explains any one of them.

Two normalisations appear and each is used in its own place.

- The closed formulas and the families use the Euclidean measure of the lecture with the $\pi^{D/2}$ taken out, $\int d^D k / \pi^{D/2}$ with propagators $k^2 + m^2$.
- **A0**, **B0**, **C0**, **D0** follow Package-X (Minkowski space, $\mu^{2\varepsilon} e^{\varepsilon\gamma_E} \int d^D k / (i\pi^{D/2})$), so that their numbers can be compared with Package-X and LoopTools.

Every feynsage function explains itself. `info(f)` prints what it computes and what its output means and most functions also take `explain=True`. Sage's own functions are explained with `f?` in a notebook.

3 Part 1: the tadpole

The lecture started from the simplest loop integral, the tadpole, in Euclidean signature,

$$I_1 = \int \frac{d^D k}{k^2 + m^2}$$

In $D = 4$ it diverges at large k (ultraviolet). In dimensional regularisation $D = 2\omega$ (Ramond) or $D = 4 - 2\varepsilon$ and the angular integral gives the solid angle of D dimensions.

3.1 The solid angle

From the Gaussian integral, $\Omega_n = 2\pi^{n/2}/\Gamma(n/2)$. Sage handles such formulas directly:

SAGE FUNCTION `var('n'), expr.subs(n=4)`

`var` makes a symbol (here n). Formulas built from symbols stay exact. `subs` puts in values. A list in square brackets evaluates the formula several times.

For $n = 2, 3, 4$ it must give the circle, the sphere and the 3-sphere:

```
In [2]: n = var('n')
Omega = 2*pi^(n/2)/gamma(n/2)                # from the Gaussian integral
[Omega.subs(n=k) for k in [2, 3, 4]]
```

```
Out[2]: [2*pi, 4*pi, 2*pi^2]
```

3.2 The tadpole with n powers

With $k = m \tan t$ the radial integral is a Beta function and

$$I_n = \int \frac{d^D k}{(k^2 + m^2)^n} = \frac{\pi^\omega \Gamma(n - \omega)}{\Gamma(n) (m^2)^{n-\omega}}$$

FEYNSAGE FUNCTION `tadpole(n, m2)`

The one-loop tadpole $\int d^D k / \pi^{D/2} (k^2 + m^2)^{-n}$ in closed form, $\Gamma(n - D/2) (m^2)^{D/2-n} / \Gamma(n)$.

- n : the power of the propagator (a number or a symbol)
- $m2$: the mass squared
- returns a Sage expression in the symbol **D** (the dimension). The $\pi^{D/2}$ is taken out of the measure, so the lecture's I_n is this times π^ω .

```
In [3]: m2 = var('m2')
tadpole(n, m2)                                # Int d^Dk/pi^(D/2) 1/(k^2 + m^2)^n
```

```
Out[3]: m2^(1/2 D - n) Gamma(-1/2 D + n)
          Gamma(n)
```

The lecture also showed that I_n follows from I_1 by differentiating with respect to m^2 ,

$$I_n(m^2) = \frac{(-1)^{n-1}}{(n-1)!} \left(\frac{d}{dm^2} \right)^{n-1} I_1(m^2)$$

SAGE FUNCTION `diff(f, x, k), f.simplify_full()`

`diff` differentiates k times with respect to x . `simplify_full` brings an expression to a simple form, so a difference that is zero prints as 0.

```
In [4]: for k in [2, 3, 4]:
        lhs = tadpole(k, m2)
        rhs = (-1)^(k-1)/factorial(k-1) * diff(tadpole(1, m2), m2, k-1)
        print("n = %d: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 =" % k,
              (lhs - rhs).simplify_full())

n = 2: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 = 0
n = 3: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 = 0
n = 4: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 = 0
```

3.3 The pole and the $\lambda\varphi^4$ tadpole

Near $D = 4$ the Gamma function has a pole.

FEYNSAGE FUNCTION `expand_eps(expr, order=0, loops=1)`

Puts $D = 4 - 2\epsilon$ into an expression with Gamma functions, multiplies by $e^{L\epsilon\gamma_E}$ ($L = \text{loops}$, the usual convention that removes γ_E) and expands in ϵ up to ϵ^{order} . Gamma functions at negative integers are rewritten first, so the poles come out correctly.

```
In [5]: expand_eps(tadpole(1, m2), 0)
```

```
Out[5]:
```

$$\frac{\epsilon m_2 \log(m_2) - (\epsilon + 1)m_2}{\epsilon}$$

This reads $-m^2/\epsilon - m^2 + m^2 \log m^2$, a simple pole as the lecture said. In $\lambda\varphi^4$ theory (Ramond, Section 4.4, eq. (4.4.3)) the tadpole diagram is $\frac{1}{2}(-\lambda)(\mu^2)^{2-\omega} \int d^{2\omega}\ell / (2\pi)^{2\omega} 1/(\ell^2 + m^2)$. Put in the integral, expand around $\omega = 2$ with $e = 2 - \omega$ and compare with the lecture's result

$$\frac{\lambda m^2}{32\pi^2} \left\{ \frac{1}{2-\omega} + \psi(2) + \ln\left(\frac{4\pi\mu^2}{m^2}\right) + O(2-\omega) \right\}$$

SAGE FUNCTION `expr.series(e, k).truncate()`

The Laurent series of `expr` in `e` up to (not including) e^k , as an ordinary expression.

```
In [6]: lam = var('lam', latex_name=r'\lambda'); mu2 = var('mu2', latex_name=r'\mu^2')
w, e = var('omega e')
# (1/2)(-lambda)(mu^2)^(2-w) Int d^(2w)l/(2 pi)^(2w) 1/(l^2 + m^2),
# the integral done as above
T = 1/2*(-lam)*mu2^(2-w) * pi^w*gamma(1-w)*m2^(w-1)/(2*pi)^(2*w)
Texp = T.subs({w: 2-e}).series(e, 1).truncate() # e = 2 - omega
lecture = lam*m2/(32*pi^2)*(1/e + psi(2) + log(4*pi*mu2/m2))
gap = (Texp - lecture).canonicalize_radical().simplify_full()
print("minus the lecture's result:", gap)
Texp.collect(e)
```

minus the lecture's result: 0

```
Out[6]:
```

$$-\frac{(\gamma_E - 2\log(2) - 1)\lambda m_2}{32\pi^2} + \frac{\lambda m_2 \log(\pi)}{32\pi^2} - \frac{\lambda m_2 \log(m_2)}{32\pi^2} + \frac{\lambda m_2 \log(\mu^2)}{32\pi^2} + \frac{\lambda m_2}{32\pi^2 e}$$

The difference is exactly zero. The digamma function at 2 is $\psi(2) = 1 - \gamma_E$:

```
In [7]: psi(2), psi(2).n()
```

```
Out[7]: | (-gamma_E + 1, 0.422784335098467)
```

4 Part 1: integration by parts for the tadpole

4.1 Integral families

An **integral family** is a list of propagators $D_1, D_2, \dots$ together with all integrals $\int 1/(D_1^{a_1} D_2^{a_2} \dots)$ with integer powers. IBP relates its members. For the tadpole there is one propagator and the members are $T(n) = \int 1/(k^2 + m^2)^n$.

FEYNSAGE FUNCTION `family(props, kin=None, loops=None, euclidean=False, name="F")`

Builds an integral family from strings.

- `props`: the propagators, each a momentum "k" or ("momentum", "mass") for a massive line
- `kin`: the scalar products of the external momenta, for example {"p^2": "s"}
- `loops`: the loop momenta (guessed from the names if left out)
- `name`: the letter used for its members, "F" if left out. Here "T" for the tadpoles and later "J" for the bubbles, as in the lecture. It is only a label.
- `euclidean=True`: propagators $q^2 + m^2$ as in the lecture (default Minkowski, $q^2 - m^2$)
- returns a `Family`. A member is written with the family's name and the powers of the propagators. For example $T(2)$ is $\int d^D k / \pi^{D/2} 1/(k^2 + m^2)^2$ and for a family with two lines $J(2,1)$ is $\int 1/(D_1^2 D_2)$. A power 0 means the line is absent and a negative power is a numerator. (This F or T or J has nothing to do with the hypergeometric function.) `.info()` prints the family, `.ibp(a)` gives the IBP identities, `.UF()` the graph polynomials.

```
In [8]: tad = family(["k", "m"], euclidean=True, name="T")
        tad.info()
```

```
Integral family 'T'
  loop momenta: k   external momenta: none   Euclidean metric
  D1 = (k)^2 + m^2
  T(a1,...,at) = Int prod_r d^d l_r prod_i D_i^(-a_i), a_i <= 0 is a numerator.
```

4.2 The IBP identity

This is Gauss's law in D dimensions. The integral of a total derivative vanishes,

$$\int d^D k \frac{\partial}{\partial k^\mu} \frac{k^\mu}{(k^2 + m^2)^n} = 0 \Rightarrow 2(\omega - n)I_n + 2nm^2 I_{n+1} = 0$$

FEYNSAGE FUNCTION `fam.ibp(a)`

All IBP identities with the **seed** `a` (a tuple of powers, here $(n,)$ for $T(n)$). For every loop momentum ℓ and every vector v (loop or external) it gives the identity from $\int \partial/\partial \ell^\mu (v^\mu \text{integrand}) = 0$, worked out exactly as in the lecture (derivative, then every scalar product written through the propagators). It returns a list of dictionaries `{powers: coefficient}`. For example `{(1,): d - 2, (2,): 2*m^2}` means $(d - 2)T(1) + 2m^2 T(2) = 0$. `d` is the dimension.

```
In [9]: for k in [1, 2, 3]:
        print("seed T(%d):" % k, tad.ibp((k,)))           # {(n,): c} means c T(n)

seed T(1): [{(1,): d - 2, (2,): 2*m^2}]
seed T(2): [{(2,): d - 4, (3,): 4*m^2}]
seed T(3): [{(3,): d - 6, (4,): 6*m^2}]
```

The first line reads $(d - 2)T(1) + 2m^2 T(2) = 0$, the identity of the lecture with $n = 1$. In general $(d - 2n)T(n) + 2nm^2 T(n + 1) = 0$.

4.3 Solving the identities: reduction

Used again and again, the identities bring every $T(n)$ down to $T(1)$. That is what a reduction program does.

FEYNSAGE FUNCTION `ibp_reduce(fam, targets, method="auto")`

Writes every target integral as a combination of **master integrals**, the members that the identities cannot remove.

- **targets**: the integrals to reduce, by their powers, for example `["T(2)", "T(3)"]` or the same as tuples `[(2,), (3,)]`
- it chooses which seeds to use, finds the symmetries of the family, writes all IBP identities and solves them (Laporta's method). `method = "ff"` (finite fields, fast), `"trimmed"` or `"exact"` (exact rational functions). `"auto"` picks one.
- returns a `Reduction`. `r[target]` is the dictionary `{master: coefficient}`, `r.masters` the list of masters, `r.info()` a summary. In a notebook it prints as typeset equations.

```
In [10]: r = ibp_reduce(tad, ["T(2)", "T(3)", "T(4)"])
         r
```

Out [10]:

$$\begin{aligned} T(2) &= -\frac{(d-2)}{2m^2} T(1) \\ T(3) &= \frac{(d-4)(d-2)}{8m^4} T(1) \\ T(4) &= -\frac{(d-6)(d-4)(d-2)}{48m^6} T(1) \end{aligned}$$

The letters can also be drawn as pictures. `draw` puts the graph of each integral in place of its name. A dot on a line is one extra power of that propagator, so $T(2)$ is the tadpole with one dot.

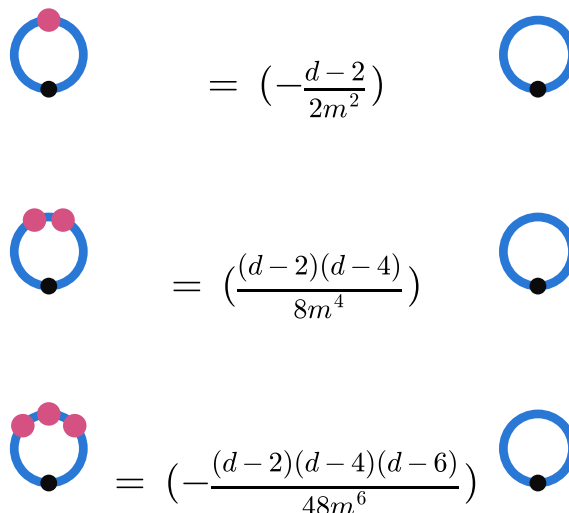
FEYNSAGE FUNCTION `r.draw(graph, targets=None, rename=None, size=1.5)`

Draws the reduction `r` as equations between Feynman diagrams, one row for each target.

- **graph**: the diagram of the family, made with `graph(...)` (Part 2 explains it). Its lines must be in the same order as the propagators of the family.
- **targets**: which rows to draw (all of them by default)
- **rename**: a dictionary to print a variable under another name, for example `{"kk": "k^2"}`
- a line with power 0 is contracted to a point, so the bubble with one line removed is drawn as a tadpole

```
In [11]: fig = r.draw(graph("A-A:m", {}), size=1.0) # the tadpole: one line from A back to A
```

Out [11]:



$$\begin{aligned} \text{Diagram with 1 dot} &= \left(-\frac{d-2}{2m^2}\right) \text{Diagram with 1 dot} \\ \text{Diagram with 2 dots} &= \left(\frac{(d-2)(d-4)}{8m^4}\right) \text{Diagram with 1 dot} \\ \text{Diagram with 3 dots} &= \left(-\frac{(d-2)(d-4)(d-6)}{48m^6}\right) \text{Diagram with 1 dot} \end{aligned}$$

The ratios must be exactly those of the Gamma functions above. The coefficients come back as rational functions of d . `SR(str(...))` turns one into an ordinary Sage expression so that it can be compared with `tadpole`:

```
In [12]: mm = var('m')
         for k in [2, 3, 4]:
             c = SR(str(r[(k,)][(1,)])) .subs({SR.var('d'): D})
             print("T(%d)/T(1) from IBP equals the Gamma-function ratio:" % k,
                   bool((c - tadpole(k, mm^2)/tadpole(1, mm^2)).simplify_full() == 0))

T(2)/T(1) from IBP equals the Gamma-function ratio: True
T(3)/T(1) from IBP equals the Gamma-function ratio: True
T(4)/T(1) from IBP equals the Gamma-function ratio: True
```

4.4 The difference equation and its solution

Argeri and Mastrolia write the identity for $U(n) = \pi^{-D/2} \int d^D k / (k^2 + 1)^n$ as a difference equation, $-(n - D/2)U(n) + nU(n+1) = 0$. Mathematica's `RSolve` solves it once $U(1)$ is given. The solution is $U(n) = \Gamma(n - D/2) / (\Gamma(1 - D/2)\Gamma(n)) U(1)$. Put it back:

```
In [13]: Uformula = lambda n: gamma(n - D/2)/(gamma(1 - D/2)*gamma(n)) # times U(1)
         (-(n - D/2)*Uformula(n) + n*Uformula(n + 1)).simplify_full()
```

```
Out[13]: 0
```

The lecture also solved it with a **factorial series**, $U(n) = \int_0^1 dt t^{n-1} v(t)$. Putting this into the difference equation and integrating by parts (with $v(1) = 0$) gives a first-order differential equation for v ,

$$\frac{v'(t)}{v(t)} = \frac{D/2 - t}{t(t-1)} \Rightarrow v(t) = v_0 t^{-D/2} (1-t)^{D/2-1}$$

and the asymptotic estimate fixes $v_0 = 1/\Gamma(D/2)$.

SAGE FUNCTION `desolve(eq, y, ivar=t), numerical_integral(f, a, b)`
`desolve` solves an ordinary differential equation for the function y of t (here $_C$ is the free constant v_0).
`numerical_integral` integrates numerically and returns the value and an error estimate. `[0]` takes the value.

Sage solves the equation and the integral gives back $U(n)$ (here $D = 2.6$, $n = 3$):

```
In [14]: t = var('t')
         v = function('v')(t)
         sol = desolve(diff(v, t) == (D/2 - t)/(t*(t - 1))*v, v, ivar=t)
         print("v(t) =", sol)
         Dv, nv = 2.6, 3
         num = numerical_integral(t^(nv - 1) * t^(-Dv/2)*(1 - t)^(Dv/2 - 1) / gamma(Dv/2), 0, 1)[0]
         print("Int_0^1 t^(n-1) v(t) dt with v0 = 1/Gamma(D/2):", num)
         print("U(n) = Gamma(n - D/2)/Gamma(n):", tadpole(nv, 1).subs(D=Dv).n())

v(t) = _C*e^(1/2*(D - 2)*log(t - 1) - 1/2*D*log(t))
Int_0^1 t^(n-1) v(t) dt with v0 = 1/Gamma(D/2): 0.4543193697876147
U(n) = Gamma(n - D/2)/Gamma(n): 0.454319366426645
```

5 Part 1: the basis of one-loop integrals

Passarino and Veltman showed that every one-loop integral is a combination of four scalar functions, A (tadpole), B (bubble), C (triangle) and D (box). Package-X computes them in Mathematica and FeynCalc uses it through FeynHelpers. FIRE and Kira do the IBP reduction for many loops.

FEYNSAGE FUNCTION `A0(m)`, `B0(s, m1, m2)`, `C0(s1, s12, s2, m0, m1, m2)`, `D0(...)`

The four scalar one-loop functions in the Package-X normalisation (Minkowski space). `B0(s, m1, m2)` has propagators (ℓ, m_1) and $(\ell + p, m_2)$ with $p^2 = s$. A_0 and B_0 come back as formulas with the pole $1/\varepsilon$ and the scale μ . A finite C_0 or D_0 stays a symbol until `.n()` asks for its number (more than 30 digits are available).

FEYNSAGE FUNCTION `finite_part(expr, mu_value=None)`, `uv_part(expr)`

The ε^0 part and the coefficient of $1/\varepsilon$ of an expression. `mu_value` sets the scale μ to a number.

```
In [15]: print("A0(1)                =", A0(1))
print("B0(5; 1, 1), finite part =", finite_part(B0(5, 1, 1), 1).n(digits=12))
print("C0(1, 2, 3; 1, 2, 3)      =", C0(1, 2, 3, 1, 2, 3).n(digits=12))
print("D0(1,2,3,4; -5,-6; 1,...) =", D0(1, 2, 3, 4, -5, -6, 1, 1, 1, 1).n(digits=12))
```

```
A0(1)                = 1/eps + log(mu^2) + 1
B0(5; 1, 1), finite part = 1.56959105904 + 1.40496294621*I
C0(1, 2, 3; 1, 2, 3)      = -0.142398839895
D0(1,2,3,4; -5,-6; 1,...) = 0.272791904368
```

6 Part 1: the bubble

The lecture followed Argeri-Mastrolia (Section 6 of their review) and Ramond, with equal masses, external momentum k and loop momentum p (Argeri and Mastrolia divide the measure by $(2\pi)^{D-2}$, which only changes the overall factor),

$$J_{n_1 n_2} = \int d^D p \frac{1}{D_1^{n_1} D_2^{n_2}}, \quad D_1 = p^2 + m^2, \quad D_2 = (p - k)^2 + m^2$$

In family this is two massive propagators. The name `kk` is just k^2 . A Sage variable name cannot contain $\wedge$, so k^2 is written `kk` (and $k \cdot p$ would be `kp`):

```
In [16]: bub = family([(("p", "m"), ("p - k", "m"))], kin={"k^2": "kk"}, loops=["p"], euclidean=True,
name="J")
bub.info()
```

```
Integral family 'J'
  loop momenta: p   external momenta: k   Euclidean metric
  D1 = (p)^2 + m^2
  D2 = (p - 1*k)^2 + m^2
  J(a1,...,at) = Int prod_r d^d l_r prod_i D_i^(-a_i), a_i <= 0 is a numerator.
```

6.1 IBP for the bubble

Now there are two vectors to put inside the derivative, the internal momentum p and the external one k , so `ibp` gives two identities:

```
In [17]: for ident in bub.ibp((1, 1)):
terms = ["(%s) J(%s)" % (c, ",".join(map(str, a))) for a, c in ident.items()]
print(" + ".join(terms), "= 0")
```

```
(d - 3) J(1,1) + (2*m^2) J(2,1) + (2*m^2 + kk) J(1,2) + (-1) J(0,2) = 0
(-kk) J(2,1) + (1) J(2,0) + (kk) J(1,2) + (-1) J(0,2) = 0
```

A dot on a line means one more power of that propagator. `ibp_reduce` solves the identities and gives the two results of the lecture,

$$J_{21} = -\frac{D-3}{k^2 + 4m^2} J_{11} + \frac{1}{k^2 + 4m^2} T(2), \quad T(2) = -\frac{D-2}{2m^2} T(1)$$

where $J(0,1)$ is the tadpole $T(1)$ (line 1 absent, power 0):

```
In [18]: rb = ibp_reduce(bub, ["J(2,1)", "J(1,2)", "J(0,2)"])
rb
```

```
Out[18]:
```

$$\begin{aligned}
 J(2,1) &= -\frac{(d-3)}{(4m^2+kk)} J(1,1) \\
 &\quad -\frac{(d-2)}{2m^2(4m^2+kk)} J(0,1) \\
 J(1,2) &= -\frac{(d-3)}{(4m^2+kk)} J(1,1) \\
 &\quad -\frac{(d-2)}{2m^2(4m^2+kk)} J(0,1) \\
 J(0,2) &= -\frac{(d-2)}{2m^2} J(0,1)
 \end{aligned}$$

The coefficients are rational functions in the variables d, kk, m . `.parent().gens()` hands these variables to us, so the comparison with the lecture is exact:

```
In [19]: d_, kk_, m_ = rb[(2, 1)][(1, 1)].parent().gens()
print("dotted bubble: ", rb[(2, 1)][(1, 1)] == -(d_ - 3)/(kk_ + 4*m_^2),
      " tadpole part:", rb[(2, 1)][(0, 1)] == 1/(kk_ + 4*m_^2) * (-(d_ - 2)/(2*m_^2)))
print("dotted tadpole: ", rb[(0, 2)][(0, 1)] == -(d_ - 2)/(2*m_^2))

dotted bubble: True  tadpole part: True
dotted tadpole: True
```

The same reduction as pictures. The graph is two lines between the vertices A and B , with k coming in at A and going out at B . `rename` prints kk as k^2 :

```
In [20]: gb = graph("A-B:m, A-B:m", {"A": "k", "B": "-k"}, kin={"k^2": "kk"}, euclidean=True)
fig = rb.draw(gb, targets=[(2, 1), (0, 2)], rename={"kk": "k^2"}, size=1.0)
```

```
Out[20]:
```

$$\begin{aligned}
 \text{Bubble with two external lines} &= \left(-\frac{d-3}{4m^2+k^2}\right) \text{Bubble with two external lines} + \left(-\frac{d-2}{2(4m^2+k^2)m^2}\right) \text{Tadpole} \\
 \text{Tadpole} &= \left(-\frac{d-2}{2m^2}\right) \text{Bubble}
 \end{aligned}$$

6.2 The differential equation

J depends on k only through k^2 , so $k_\mu \partial J / \partial k_\mu = 2k^2 \partial J / \partial k^2$. Differentiating under the integral,

$$k_\mu \frac{\partial J}{\partial k_\mu} = T(2) - J_{11} - k^2 J_{12}$$

The reduction above already contains J_{12} and $T(2)$ in terms of the masters, so the right-hand side becomes $AJ + BT(1)$. That is the differential equation. Compare it with the lecture,

$$\frac{dJ}{dk^2} + \frac{1}{2} \left[\frac{1}{k^2} - \frac{D-3}{k^2+4m^2} \right] J = -\frac{D-2}{4m^2} \left[\frac{1}{k^2} - \frac{1}{k^2+4m^2} \right] T(1)$$

```
In [21]: # k.dJ/dk = J(0,2) - J(1,1) - k^2 J(1,2) and dJ/dk^2 = (k.dJ/dk)/(2 k^2);
# reduce the right side
kk = SR.var('kk'); dd = SR.var('d'); m = SR.var('m')
c = lambda a, mm_: SR(str(rb[a].get(mm_, 0)))
A = (c((0, 2), (1, 1)) - 1 - kk*c((1, 2), (1, 1)))/(2*kk) # coefficient of J(1,1)
B = (c((0, 2), (0, 1)) - kk*c((1, 2), (0, 1)))/(2*kk) # coefficient of T(1)
print("dJ/dk^2 = A J + B T(1) with")
print("  A =", A.factor())
print("  B =", B.factor())
print("as in the notes:",
      bool((A + (1/kk - (dd - 3)/(kk + 4*m^2))/2).simplify_full() == 0),
      bool((B + (dd - 2)/(4*m^2)*(1/kk - 1/(kk + 4*m^2))).simplify_full() == 0))

dJ/dk^2 = A J + B T(1) with
A = 1/2*(d*kk - 4*m^2 - 4*kk)/((4*m^2 + kk)*kk)
B = -(d - 2)/((4*m^2 + kk)*kk)
as in the notes: True True
```

6.3 Solving the differential equation

Now we solve it, with its boundary condition. Put $k^2 = 4m^2x$, so that $dJ/dx = 4m^2 dJ/dk^2$. `partial_fraction` splits the coefficients into simple poles. For finite x the equation is singular only at $x = 0$ ($k^2 = 0$) and $x = -1$ (the threshold $k^2 = -4m^2$ in this Euclidean metric):

```
In [22]: x = var('x')
# write k^2 = 4 m^2 x; then dJ/dx = 4 m^2 dJ/dk^2
Ax = (4*m^2*A).subs(kk=4*m^2*x).subs({dd: D}).simplify_full()
Bx = (4*m^2*B).subs(kk=4*m^2*x).subs({dd: D}).simplify_full()
print("dJ/dx = (%s) J" % Ax, partial_fraction(x))
print("          + (%s) T(1)" % Bx, partial_fraction(x))

dJ/dx = (1/2*(D - 3)/(x + 1) - 1/2/x) J
        + (1/4*(D - 2)/(m^2*(x + 1)) - 1/4*(D - 2)/(m^2*x)) T(1)
```

SAGE FUNCTION `expr.subs(kk=...)`, `expr.subs({dd: D})`, `expr.partial_fraction(x)`
`subs` replaces a variable. The keyword form `subs(kk=...)` works when the Python name and the Sage name are the same. `dd` is the Sage variable printed `d`, so it goes in a dictionary. `partial_fraction(x)` writes a rational function of x as a sum of simple fractions.

The homogeneous equation. `desolve` gives $J_0 = C x^{-1/2} (1+x)^{(D-3)/2}$:

```
In [23]: J0 = function('J0')(x)
hom = desolve(diff(J0, x) == Ax*J0, J0, ivar=x) # the homogeneous equation
hom.canonicalize_radical() # write e^(a log u) as u^a
```

```
Out[23]:
```

$$\frac{C(x+1)^{\frac{1}{2}D-\frac{3}{2}}}{\sqrt{x}}$$

The boundary condition. At $k^2 = 0$ the bubble is $J(D, 0) = \int d^D p / (p^2 + m^2)^2 = T(2)$, a finite number. The homogeneous solution behaves as $x^{-1/2}$ there, so it must not appear. Try a power series $J = \sum_n c_n x^n$, which is regular at $x = 0$. Multiply the equation by $x(1+x)$ to clear the denominators and ask that every power of x cancels. The x^0 equation does not contain any free constant. It **fixes** c_0 and the value is exactly $T(2)$. So regularity at $x = 0$ and the boundary condition are the same statement. Every higher c_n then follows from the one before:

```
In [24]: N = 6
cs = [var('c%d' % i) for i in range(N + 1)]
T1 = var('T1') # the master T(1)
Js = sum(cs[i]*x^i for i in range(N + 1)) # J as a power series in x
eq = (x*(1 + x)*(diff(Js, x) - Ax*Js - Bx*T1)).simplify_rational().expand()
eqs = [eq.coefficient(x, k) == 0 for k in range(N)]
print("the x^0 equation:", eqs[0])
sol = solve(eqs, cs[:N], solution_dict=True)[0]
is_T2 = bool((sol[cs[0]] + (D - 2)/(2*m^2)*T1).simplify_full() == 0)
print("so J(D, 0) = c0 =", sol[cs[0]].factor(), " equal to T(2):", is_T2)
for n in range(4):
    ratio = (sol[cs[n + 1]]/sol[cs[n]]).factor()
    hyp = -(2 - D/2 + n)*(1 + n)/((3/2 + n)*(1 + n)) # term ratio of the 2F1
    same = bool((ratio - hyp).simplify_full() == 0)
    print("c%d/c%d =" % (n + 1, n), ratio, " same as 2F1:", same)

the x^0 equation: 1/2*c0 + 1/4*D*T1/m^2 - 1/2*T1/m^2 == 0
so J(D, 0) = c0 = -1/2*(D - 2)*T1/m^2 equal to T(2): True
c1/c0 = 1/3*D - 4/3 same as 2F1: True
c2/c1 = 1/5*D - 6/5 same as 2F1: True
c3/c2 = 1/7*D - 8/7 same as 2F1: True
c4/c3 = 1/9*D - 10/9 same as 2F1: True
```

SAGE FUNCTION `solve(eqs, vars, solution_dict=True)`, `expr.coefficient(x, k)`
`coefficient(x, k)` takes the coefficient of x^k in an expanded expression. `solve` solves a list of equations for the listed variables. With `solution_dict=True` it returns a list of dictionaries {variable: value}.

The ratio of two neighbouring terms of ${}_2F_1(a, b; c; z) = \sum_n (a)_n (b)_n / ((c)_n n!) z^n$ is $(a + n)(b + n) / ((c + n)(1 + n)) z$. The ratios c_{n+1}/c_n found above are exactly this with $a = 2 - D/2$, $b = 1$, $c = 3/2$, $z = -x$. With $c_0 = T(2) = \Gamma(2 - D/2) m^{D-4}$ the solution is

$$J(D, k^2) = \Gamma(2 - D/2) m^{D-4} {}_2F_1(2 - D/2, 1; 3/2; -k^2/4m^2)$$

FEYNSAGE FUNCTION `bubble_equal_mass(p2, m2)`
 The equal-mass bubble $\int d^D p / \pi^{D/2} 1 / ((p^2 + m^2)((p - k)^2 + m^2))$ at $k^2 = p2$, this solution of the differential equation. The kinematic variable sits in the argument and the dimension in the parameters.

A numerical check. Forget the closed form and integrate the equation with a numerical solver (SciPy's `solve_ivp`), starting next to $x = 0$ from the boundary value. The numbers agree with the hypergeometric function to all ten digits shown ($D = 3.3$, $m = 1$):

```
In [25]: from scipy.integrate import solve_ivp
Dv, mv = 3.3, 1
T1v = float(tadpole(1, mv^2).subs(D=Dv))
fA = fast_callable(Ax.subs(m=mv, D=Dv), vars=[x])
fB = fast_callable(Bx.subs(m=mv, D=Dv), vars=[x])
x0 = 1e-6 # start next to the boundary x = 0
J_start = float(sum(sol[cs[i]].subs(D=Dv, m=mv, T1=T1v)*x0^i for i in range(N)))
num = solve_ivp(lambda t, y: [fA(t)*y[0] + fB(t)*T1v], [x0, 2.0], [J_start],
                rtol=1e-11, atol=1e-13, dense_output=True)
for xv in [0.125, 0.5, 2.0]:
    exact = bubble_equal_mass(4*mv^2*xv, mv^2).subs(D=Dv).n() # Gamma(2-D/2) m^(D-4) 2F1
    print("x = %.3f solved numerically: %.10f closed form: %.10f"
          % (xv, num.sol(xv)[0], exact))

x = 0.125 solved numerically: 2.4765120121 closed form: 2.4765120121
x = 0.500 solved numerically: 2.3095817834 closed form: 2.3095817834
x = 2.000 solved numerically: 1.9299216509 closed form: 1.9299216509
```

SAGE, SCIPY FUNCTION `fast_callable(expr, vars=[x]), solve_ivp(f, [x0, x1], y0, dense_output=True)`
`fast_callable` turns a Sage expression into a fast numerical function. `solve_ivp` integrates $y' = f(x, y)$ from x_0 to x_1 with the starting value y_0 . `num.sol(x)` gives the solution at any x in between.

6.4 The hypergeometric function in Sage

Sage knows ${}_2F_1$ as `hypergeometric([a, b], [c], z)`. `.n(digits=25)` evaluates it to 25 digits (through `mpmath`, which Sage contains). Below the same number is found from the series itself and directly from `mpmath`. The parameters are exact fractions, since a decimal number like `0.35` would limit the precision to 16 digits. For special parameters Maxima can rewrite the function in elementary functions. At $D = 3$ the bubble is an arc tangent:

```
In [26]: a, b, c, z = 7/20, 1, 3/2, -1/2          # 2F1(2 - D/2, 1; 3/2; -x) at D = 3.3, x = 1/2
h = hypergeometric([a, b], [c], z)
print("Sage: ", h.n(digits=25))
poch = rising_factorial                          # (a)_n = a (a+1) ... (a+n-1)
term = lambda n: poch(a, n)*poch(b, n)/poch(c, n)*z^n/factorial(n)
print("the series: ", sum(term(n) for n in range(80)).n(digits=25))
import mpmath; mpmath.mp.dps = 25
print("mpmath.hyp2f1: ", mpmath.hyp2f1(a, b, c, z))
assume(x > 0)
h3 = hypergeometric([1/2, 1], [3/2], -x)        # D = 3
print("D = 3 in closed form:", h3.simplify_hypergeometric(algorithm='maxima'))

Sage:      0.9070889481401101364449382
the series: 0.9070889481401101364449382
mpmath.hyp2f1: 0.9070889481401101364449382
D = 3 in closed form: arctan(sqrt(x))/sqrt(x)
```

SAGE FUNCTION `hypergeometric([a, b], [c], z), rising_factorial(a, n), simplify_hypergeometric(algorithm='maxima')`
`hypergeometric` is the generalised hypergeometric function ${}_pF_q$ with the lists of upper and lower parameters. `rising_factorial(a, n)` is the Pochhammer symbol $(a)_n = a(a+1)\dots(a+n-1)$. `simplify_hypergeometric` looks for a closed form. Maxima needs to know the sign of x , so `assume(x > 0)` comes first.

7 Part 1: the fish diagram and Feynman parameters

The fish diagram of $\lambda\varphi^4$ is the same bubble. The lecture combined the two denominators with the Feynman trick,

$$\frac{1}{AB} = \int_0^1 \frac{dx}{[xA + (1-x)B]^2}$$

SAGE FUNCTION `assume(cond), integrate(f, x, a, b)`
`integrate` does definite integrals exactly. `assume` tells it what it needs to know about the symbols (here that $A, B > 0$).

```
In [27]: A_, B_, x = var('A B x')
assume(A_ > 0, B_ > 0, B_ - A_ > 0)          # any order of A and B gives the same
integrate(1/(x*A_ + (1-x)*B_)^2, x, 0, 1).factor()
```

```
Out [27]: 1
           AB
```

After the shift $\ell \rightarrow \ell + p(1-x)$ the loop integral is a tadpole with mass² $m^2 + p^2x(1-x)$ and two powers, so `tadpole(2, ...)` gives it. Only the integral over the Feynman parameter x is left. Done numerically it equals the hypergeometric result of the differential equation ($D = 3.3$, $m = 1$, $p^2 = 3$):

```
In [28]: Dv, mv, pv = 3.3, 1, 3
# after the shift l -> l + p(1 - x) the loop integral is a tadpole
# with mass^2 = m^2 + p^2 x(1 - x)
tad_x = lambda xx: tadpole(2, mv^2 + pv*xx*(1 - xx)).subs(D=Dv).n()
fp = numerical_integral(tad_x, 0, 1)[0]
print("Feynman parameter integral:", fp)
print("bubble from the DE (2F1): ", bubble_equal_mass(pv, mv^2).subs(D=Dv).n())
```

Feynman parameter integral: 2.2221997148168473
bubble from the DE (2F1): 2.22219971481685

7.1 Ramond's expansion around $D = 4$

Expanding around $\omega = 2$ the finite part is $-\int_0^1 dx \ln(1 + x(1-x)p^2/m^2)$, which Ramond (eqs. (4.4.15) and (4.4.16)) writes as $2 - \sqrt{1 + 4m^2/p^2} \ln[(\sqrt{1 + 4m^2/p^2} + 1)/(\sqrt{1 + 4m^2/p^2} - 1)]$:

```
In [29]: p2v = 3
finite = -numerical_integral(lambda xx: log(1 + xx*(1 - xx)*p2v), 0, 1)[0]
b = sqrt(1 + 4/p2v)
print("finite part, integral over x: ", finite)
print("Ramond: 2 - b ln((b + 1)/(b - 1)):", (2 - b*log((b + 1)/(b - 1))).n())
```

finite part, integral over x: -0.3933253674083756
Ramond: 2 - b ln((b + 1)/(b - 1)): -0.393325367408375

7.2 Going to Minkowski space: the imaginary part

In Minkowski space $p^2 \rightarrow -s$ and above the threshold $s > 4m^2$ the argument of the logarithm turns negative. Then $\ln z = \ln|z| + i\pi$ for $z < 0$ and the bubble gets an imaginary part. By the optical theorem it must, because above threshold the two particles in the loop can be real. $B0$ is the same bubble in Minkowski space, so its imaginary part should be $\pi\beta$ with $\beta = \sqrt{1 - 4m^2/s}$ (here $m = 1$):

```
In [30]: print("ln(-2)      =", log(-2).n())
print("ln 2 + i pi =", (log(2) + I*pi).n())
for sv in [3, 5, 10]:
    val = finite_part(B0(sv, 1, 1), 1).n(digits=12)
    beta = sqrt(1 - 4/sv) if sv > 4 else 0
    print("s = %2d (m = 1):  Im of the bubble = %.10f    pi*beta = %.10f"
          % (sv, imag(val), (pi*beta).n()))
```

ln(-2) = 0.693147180559945 + 3.14159265358979*I
ln 2 + i pi = 0.693147180559945 + 3.14159265358979*I
s = 3 (m = 1): Im of the bubble = 0.0000000000 pi*beta = 0.0000000000
s = 5 (m = 1): Im of the bubble = 1.4049629462 pi*beta = 1.4049629462
s = 10 (m = 1): Im of the bubble = 2.4334672056 pi*beta = 2.4334672056

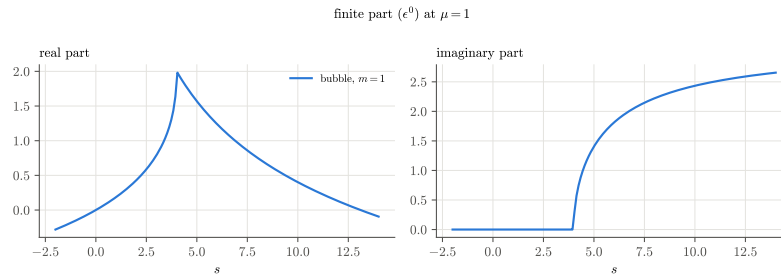
FEYNSAGE FUNCTION `quick_plot`(exprs, (s, a, b), labels=None)

Plots one or more expressions in s from a to b . For an expression with $1/\varepsilon$ it plots the finite part. Complex values are drawn as the real part (full line) and the imaginary part (dashed). Returns a matplotlib figure.

The imaginary part starts at $s = 4$:

```
In [31]: from feynsage.plotting import quick_plot, set_theme
set_theme()
s = var('s')
fig = quick_plot([B0(s, 1, 1)], (s, -2, 14), labels=["bubble, $m = 1$"])
```

Out[31]:



8 Part 2: graph polynomials

The second lecture combined the denominators of any diagram at once and read the result off the graph. Take a diagram with L loops and N propagators $D_j = q_j^2 - m_j^2 + i0$ (Minkowski space), raised to powers ν_j . Its **Feynman integral** is the integral over all loop momenta of the product of propagators. With Feynman parameters it becomes

$$\begin{aligned} I(\nu_1, \dots, \nu_N) &= \int \prod_{l=1}^L \frac{d^D \ell_l}{i\pi^{D/2}} \frac{1}{D_1^{\nu_1} \dots D_N^{\nu_N}} \\ &= (-1)^{N_\nu} \frac{\Gamma(N_\nu - LD/2)}{\prod_j \Gamma(\nu_j)} \int_0^\infty \prod_{j=1}^N dx_j x_j^{\nu_j-1} \delta\left(1 - \sum x_j\right) \frac{\mathcal{U}^{N_\nu - (L+1)D/2}}{\mathcal{F}^{N_\nu - LD/2}} \end{aligned}$$

with $N_\nu = \sum_j \nu_j$. This is not yet the scattering amplitude. The amplitude contains such integrals, multiplied by the couplings and by the numerator from the Dirac algebra (the pion in the last chapter shows how they fit together). The sign $(-1)^{N_\nu}$ comes from the Minkowski propagators. In Euclidean space, with $d^D \ell / \pi^{D/2}$ and propagators $\ell^2 + m^2$, the same formula holds without it.

The two polynomials come from the graph:

- A 1-tree (spanning tree) is what is left after erasing L lines so that the rest touches every vertex without a loop. Each gives the product of the x_i of the erased lines. $\mathcal{U}$ is their sum, homogeneous of degree L .
- A 2-tree is what is left after erasing one more line, so that the graph falls into two pieces. Each gives the product of the erased x_i times the square of the momentum flowing from one piece to the other. Their sum is V and $\mathcal{F} = V + \mathcal{U} \sum_i x_i m_i^2$.

FEYNSAGE FUNCTION `graph(edges, legs, kin=None, euclidean=False)`

A Feynman diagram as a graph, from strings.

- **edges**: the lines, "A-B" between vertices A and B, or "A-B:m" for a line of mass m . **The order of the lines is the numbering** $x_1, x_2, \dots$ (the lecture's $\alpha_1, \alpha_2, \dots$).
- **legs**: the momentum **entering** at each vertex, for example {"A": "p", "B": "-p"}. They must add up to zero.
- **kin**: the scalar products of the external momenta. **euclidean=True** for Euclidean signs.
- returns a `FeynmanGraph` with these methods:
 - `.spanning_trees()`: the 1-trees, as tuples of the **erased** lines (counted from 0)
 - `.two_forests()`: the 2-trees, as pairs (erased lines, vertices of one of the two pieces)
 - `.U()`, `.F0()` ($= V$), `.F()`: the polynomials, read off the trees exactly by the rules above
 - `.plot()`: a picture
 - `.N`, `.V`, `.L`: the numbers of lines, vertices and loops

FEYNSAGE FUNCTION `draw_panels(g, erased, titles=None, ncols=5)`

Draws the graph g several times side by side, each copy with its own erased lines (grey and dashed). Feed it `g.spanning_trees()` to see all the 1-trees.

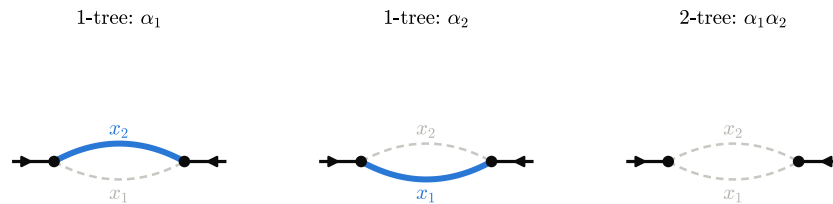
8.1 The bubble with two masses

Two 1-trees (erase the top line or the bottom line) and one 2-tree (erase both lines, then p flows between the two vertices).

```
In [32]: from feynsage.plotting import draw_panels
xs = lambda rem: "$" + "".join("\\alpha_{%d}" % (i + 1) for i in rem) + "$"
g2 = graph("A-B:m1, A-B:m2", {"A": "p", "B": "-p"}, kin={"p^2": "pp"}, euclidean=True)
trees = g2.spanning_trees(); forests = [r for r, c in g2.two_forests()]
titles = ["1-tree: " + xs(r) for r in trees] + ["2-tree: " + xs(r) for r in forests]
fig = draw_panels(g2, trees + forests, titles=titles, momenta=False, ncols=3, size=1.9)
print("U =", g2.U())
print("V =", g2.F0())
print("F = V + (m1^2 x1 + m2^2 x2) U =", g2.F())
```

```
U = x1 + x2
V = pp*x1*x2
F = V + (m1^2 x1 + m2^2 x2) U = m1^2*x1^2 + (m1^2 + m2^2 + pp)*x1*x2 + m2^2*x2^2
```

Out [32]:



So $\mathcal{U} = \alpha_1 + \alpha_2$, $V = \alpha_1 \alpha_2 p^2$ and $\mathcal{F} = V + (m_1^2 \alpha_1 + m_2^2 \alpha_2) \mathcal{U}$, as in the lecture.

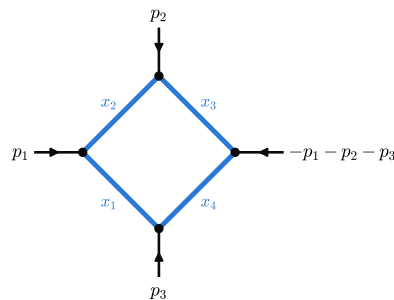
8.2 The box (Smirnov, Fig. 3.6)

The massless box with $p_i^2 = 0$, $s = (p_1 + p_2)^2$ and $t = (p_1 + p_3)^2$, with the lines numbered as in Smirnov's Fig. 3.6 (2006). Line 1 is on top (between p_1 and p_3), 2 on the left (between p_1 and p_2), 3 at the bottom (between p_2 and p_4) and 4 on the right (between p_3 and p_4). feynsage draws the square turned by 45 degrees.

```
In [33]: # Smirnov (2006) Fig. 3.6: line 1 on top, 2 on the left, 3 at the bottom, 4 on the right;
# p1 and p2 enter on the left, p3 and p4 on the right, s = (p1 + p2)^2, t = (p1 + p3)^2
box = graph("TL-TR, TL-BL, BL-BR, TR-BR",
            {"TL": "p1", "BL": "p2", "TR": "p3", "BR": "-p1-p2-p3"},
            kin={"p1^2": 0, "p2^2": 0, "p3^2": 0,
                "p1.p2": "s/2", "p1.p3": "t/2", "p2.p3": "-(s+t)/2"},
            euclidean=True)
fig = box.plot(figsize=(2.8, 2.8))
print("U =", box.U(), " V =", box.F0())
```

```
U = x1 + x2 + x3 + x4      V = s*x1*x3 + t*x2*x4
```

Out [33]:

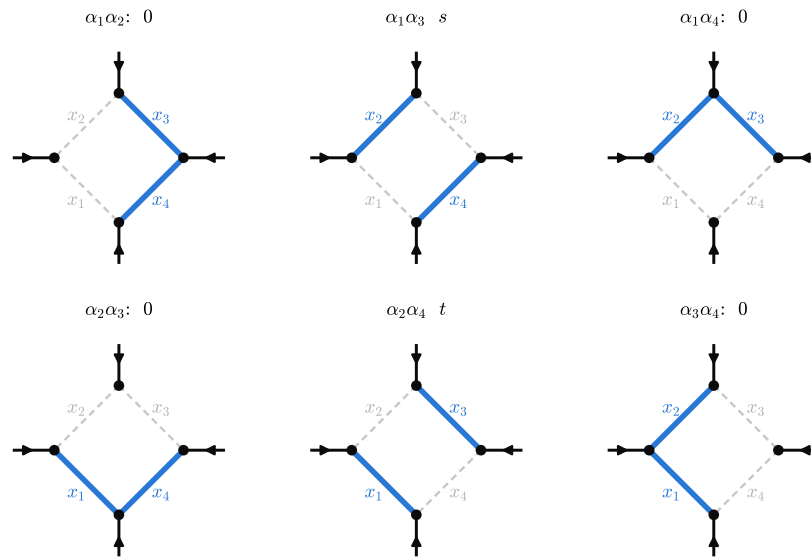


All six 2-trees. Only the two that cut the box into a left and a right half, or a top and a bottom half, carry momentum. The other four cut off a single corner, which receives only one p_i with $p_i^2 = 0$, so they do not contribute (the grey pictures of the lecture). The small helper `momentum_in` adds up the momenta entering one piece:

```
In [34]: def momentum_in(g, comp):
    tot = {}
    for v in comp:
        for k, c in g.ext.get(v, {}).items():
            tot[k] = tot.get(k, 0) + c
    return {k: c for k, c in tot.items() if c}

ft = box.two_forests()
titles = []
for rem, comp in ft:
    P2 = box.kin.square_external(momentum_in(box, comp))
    titles.append(xs(rem) + (": 0" if P2 == 0 else " $s$" % latex(P2)))
fig = draw_panels(box, [r for r, c in ft], titles=titles, momenta=False, ncols=3,
    size=2.0)
```

Out[34]:



THE LABELS MATTER

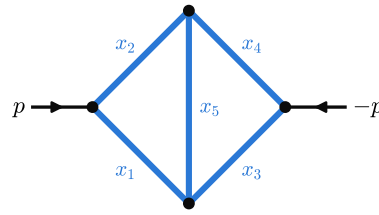
Erasing the top and bottom lines 1 and 3 separates p_1, p_2 from p_3, p_4 and gives $s \alpha_1 \alpha_3$. Erasing the left and right lines 2 and 4 gives $t \alpha_2 \alpha_4$. So $V = s \alpha_1 \alpha_3 + t \alpha_2 \alpha_4$, as on the board and in Smirnov's eq. (3.28). His eq. (3.27) prints s and t the other way round, a misprint that does no harm when all powers are 1. The safe rule is to look at the picture. s goes with the pair of lines whose removal separates p_1, p_2 from p_3, p_4 .

8.3 Smirnov's two-loop example

This is the two-loop propagator diagram of the lecture, Smirnov's Fig. 3.10 and Example 3.5 (2006), with the lines numbered as in the book. Lines 1 and 2 are on the left, 3 and 4 on the right and 5 in the middle. The polynomials below are his eqs. (3.44) and (3.45).

```
In [35]: kite = graph("L-B, L-T, B-R, T-R, T-B", {"L": "p", "R": "-p"}, kin={"p^2": "pp"},
    euclidean=True)
fig = kite.plot(figsize=(2.8, 2.8))
```

Out[35]:

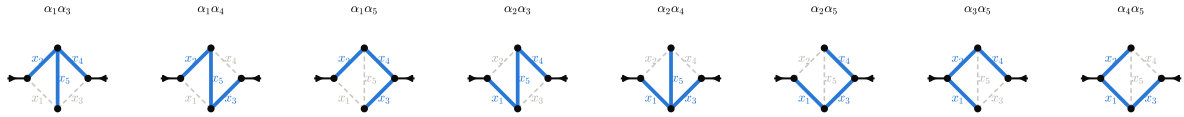


Eight 1-trees, each erasing two lines, give $\mathcal{U}$ of degree 2:

```
In [36]: trees = kite.spanning_trees()
fig = draw_panels(kite, trees, titles=[xs(r) for r in trees], momenta=False, ncols=8,
                 size=1.5)
print(len(trees), "1-trees: U =", kite.U())
```

8 1-trees: $U = x1*x3 + x2*x3 + x1*x4 + x2*x4 + x1*x5 + x2*x5 + x3*x5 + x4*x5$

Out[36]:

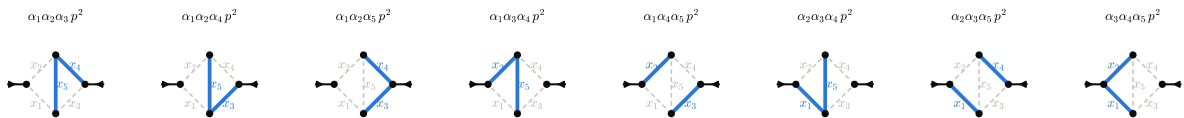


Eight 2-trees carry the momentum p , each erasing three lines:

```
In [37]: carry = [r for r, c in kite.two_forests() if momentum_in(kite, c)]
fig = draw_panels(kite, carry, titles=[xs(r) + "$\\,p^2$" for r in carry], momenta=False,
                 ncols=8, size=1.5)
print(len(carry), "2-trees with p^2:")
terms = str(kite.F()).split(" + ")
print("F =", " + ".join(terms[:4]))
print(" +", " + ".join(terms[4:]))
```

8 2-trees with p^2 :
 $F = pp*x1*x2*x3 + pp*x1*x2*x4 + pp*x1*x3*x4 + pp*x2*x3*x4$
 $+ pp*x1*x2*x5 + pp*x2*x3*x5 + pp*x1*x4*x5 + pp*x3*x4*x5$

Out[37]:



These are the 1-trees and 2-trees drawn in the lecture.

8.4 From $\mathcal{U}$ and $\mathcal{F}$ back to the integral

Put $\mathcal{U}$ and $\mathcal{F}$ into the general formula. For the massless bubble ($N = 2$, $L = 1$) the delta function leaves one integral over x . `SR(str(...))` turns the polynomials into ordinary expressions so that $x_1 = x$, $x_2 = 1 - x$ can be put in. At $D = 3.3$ and $p^2 = 1$ the result agrees with the closed form `bubble_massless(1, 1, 1)` (to the precision of the numerical integration):

```
In [38]: # the massless bubble from U and F with the general formula:
# N = 2 lines, L = 1 loop, powers 1
from feynsage.oneloop import bubble_massless
g0 = graph("A-B, A-B", {"A": "p", "B": "-p"}, kin={"p^2": 1}, euclidean=True)
U0, F0 = g0.U(), g0.F()
Dv, N, L = 3.3, 2, 1
xx = var('xx')
u = SR(str(U0)).subs(x1=xx, x2=1 - xx); f = SR(str(F0)).subs(x1=xx, x2=1 - xx)
val = gamma(N - L*Dv/2) * numerical_integral(u^(N - (L + 1)*Dv/2)/f^(N - L*Dv/2), 0, 1)[0]
print("U =", U0, " F =", F0)
print("from U and F:", val, " closed form:", bubble_massless(1, 1, 1).subs(D=Dv).n())

U = x1 + x2      F = x1*x2
from U and F: 5.44044255456555      closed form: 5.44044257084738
```

8.5 Sir's notebook: completing the square

The Mathematica notebook shown in the lecture finds $\mathcal{U}$ and $\mathcal{F}$ in a second way, without drawing any trees. The formula it uses was not written on the board, so here it is step by step.

Step 1: one denominator. With Feynman parameters x_i all the propagators are combined into one sum, $\sum_i x_i D_i$. Each D_i is quadratic in the loop momentum, so the sum is too.

Step 2: sort it by powers of the loop momentum. For one loop with loop momentum ℓ every quadratic expression can be written as

$$\sum_i x_i D_i = M \ell^2 - 2Q \cdot \ell + J$$

- M collects everything in front of ℓ^2 . Every propagator contains ℓ^2 once, so $M = \sum_i x_i$.
- Q is a momentum. It collects the terms linear in ℓ (the factor -2 is a convention that makes the next step neat).
- J is the rest, made of external momenta and masses with no ℓ at all.

Step 3: complete the square. Shift $\ell = \ell' + Q/M$:

$$M \ell^2 - 2Q \cdot \ell + J = M \ell'^2 + (J - (Q \cdot Q)/M)$$

Now the integral over ℓ' is a tadpole, because the loop momentum appears only as ℓ'^2 , with a “mass” $(J - Q^2/M)/M$. Doing it gives a power of M and a power of $J - Q^2/M$ and they combine exactly into the general formula above with

$$\mathcal{U} = M = \det M, \quad \mathcal{F} = \det M (J - Q M^{-1} Q)$$

At more loops there are several loop momenta $\ell_1, \ell_2, \dots$. The coefficients of $\ell_r \cdot \ell_s$ form a matrix M (2 by 2 at two loops) and Q becomes a list of momenta (one per loop). The same formulas hold with $\det M$ and the matrix inverse M^{-1} . This is why the notebook writes them with a determinant.

As a worked example take the bubble with two masses (Euclidean, $D_1 = k^2 + m_1^2$, $D_2 = (k + p)^2 + m_2^2$). Write the scalar products as symbols ($kk = k^2$, $kp = k \cdot p$, $pp = p^2$), expand and read off M , Q and J as the coefficients.

```
In [39]: # the two-mass bubble, Euclidean: D1 = k^2 + m1^2, D2 = (k + p)^2 + m2^2
# write the scalar products as symbols: kk = k.k, kp = k.p, pp = p.p
x1, x2, m1, m2, kk, kp, pp = var('x1 x2 m1 m2 kk kp pp')
S = (x1*(kk + m1^2) + x2*(kk + 2*kp + pp + m2^2)).expand()
M = S.coefficient(kk)
Qc = -S.coefficient(kp)/2
J = S.subs(kk=0, kp=0)
print("sum x_i D_i =", S)
print("M =", M, " Q = (%s) p" % Qc, " J =", J)
U_sq = M
# det M (J - Q.M^-1.Q), with Q.Q = Qc^2 p.p
F_sq = (M*(J - Qc^2*pp/M)).simplify_rational().expand()
print("U = det M =", U_sq)
print("F = det M (J - Q M^-1 Q) =", F_sq.collect(pp))

# sum_i x_i D_i
# the coefficient of k.k
# -2 Q.k: Q = Qc * p
# what has no loop momentum
```

```

sum x_i D_i = m1^2*x1 + m2^2*x2 + kk*x1 + kk*x2 + 2*kp*x2 + pp*x2
M = x1 + x2      Q = (-x2) p      J = m1^2*x1 + m2^2*x2 + pp*x2
U = det M = x1 + x2
F = det M (J - Q M^-1 Q) = m1^2*x1^2 + m1^2*x1*x2 + m2^2*x1*x2 + m2^2*x2^2 + pp*x1*x2

```

This is the same $\mathcal{F}$ as from the 1-trees and 2-trees above.

FEYNSAGE FUNCTION `fam.UF()`

$\mathcal{U}$ and $\mathcal{F}$ of a family by exactly this method. It builds M , Q and J from the list of propagators and returns $\det M$ and $\det M(J - QM^{-1}Q)$, for any number of loops (no graph needed). For a Minkowski family the signs are those of the notebook.

```

In [40]: fam2 = family(["k", "m1"), ("k + p", "m2"), kin={"p^2": "pp"}, euclidean=True)
U_f, F_f = fam2.UF()
print("fam.UF() gives the same:      ", bool(SR(str(U_f)) == U_sq),
      bool((SR(str(F_f)) - F_sq).expand() == 0))
print("and the 2-trees gave the same:", bool((SR(str(g2.F())) - F_sq).expand() == 0))

fam.UF() gives the same:      True True
and the 2-trees gave the same: True

```

The notebook's own examples, the triangle, the bubble with two masses and the box (Minkowski signs, as in the notebook), come out as on the screen in the lecture:

```

In [41]: # sir's notebook completes the square, sum x_i D_i = l.M.l - 2 Q.l + J:
# U = det M, F = det M (J - Q.M^-1.Q)
tri = family(["k", "k + p1", "k + p1 + p2"],
             kin={"p1^2": "P1Sq", "p2^2": "P2Sq", "p1.p2": "(Q5q - P1Sq - P2Sq)/2"})
bb = family(["k", "m1"), ("k + p1", "m2"), kin={"p1^2": "p1sq"})
bx = family(["k", "k + p1", "k + p1 + p2", "k - p3"],
             kin={"p1^2": 0, "p2^2": 0, "p3^2": 0,
                  "p1.p2": "s/2", "p1.p3": "t/2", "p2.p3": "-(s+t)/2"})
for name, fam in [("triangle", tri), ("bubble", bb), ("box", bx)]:
    U_, F_ = fam.UF()
    print("%-9s U = %-18s F = %s" % (name, U_, F_))

triangle U = x1 + x2 + x3      F = (-P1Sq)*x1*x2 + (-Q5q)*x1*x3 + (-P2Sq)*x2*x3
bubble   U = x1 + x2          F = m1^2*x1^2 + (m1^2 + m2^2 - p1sq)*x1*x2 + m2^2*x2^2
box      U = x1 + x2 + x3 + x4 F = (-s)*x1*x3 + (-t)*x2*x4

```

9 A real prediction: the lifetime of the neutral pion

Now put the tools together for a number that is measured in a laboratory. The neutral pion lives for about 8.4×10^{-17} s and in 98.8% of the cases decays into two photons. It has no charge, so the photons cannot attach to it directly. The decay goes through a **triangle** of charged quarks. The physics is explained below as we go and every step is done with feynsage, with the graph method of Part 2 at its centre.

THE MODEL

The quarks u and d ($Q_u = 2/3$, $Q_d = -1/3$) come in N_c colours. They couple to the photon with charge Qe and to the pion through $g\bar{q}\gamma_5\tau_3q\pi^0$ ($\tau_3 = +1$ for u , -1 for d), with $g = m/f_\pi$ (the Goldberger-Treiman relation, $f_\pi \approx 92$ MeV). For each quark there are two triangle diagrams, one for each order of the photons. The photons have $k_1^2 = k_2^2 = 0$ and the pion $(k_1 + k_2)^2 = m_\pi^2$.

9.1 Step 1: the graph, its trees and its polynomials

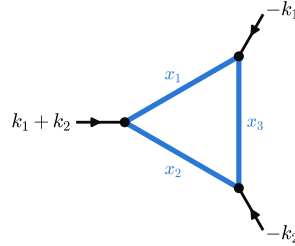
Write the triangle as a graph. The pion vertex is P and the photon vertices are A and B. All three lines are quarks of mass m . Line 1 is P-A (parameter x_1), line 2 is B-P (x_2) and line 3 is A-B (x_3). The pion brings in $k_1 + k_2$ at P and the photons take out k_1 at A and k_2 at B (Minkowski space, $k_1^2 = k_2^2 = 0$, $2k_1 \cdot k_2 = m_\pi^2$):

```
In [42]: # the quark triangle: P = pion vertex, A and B = photon vertices;
# all three lines are quarks of mass mq
# line 1 = P-A (Feynman parameter x), 2 = B-P (y), 3 = A-B (1 - x - y); Minkowski signs
tri = graph("P-A:mq, B-P:mq, A-B:mq", {"P": "k1 + k2", "A": "-k1", "B": "-k2"},
            kin={"k1^2": 0, "k2^2": 0, "k1.k2": "mpi2/2"})
fig = tri.plot(figsize=(2.6, 2.6))
print("U =", tri.U())
print("F =", tri.F())
```

$$U = x_1 + x_2 + x_3$$

$$F = mq^2 x_1^2 + (2mq^2 - mpi^2)x_1 x_2 + mq^2 x_2^2 + 2mq^2 x_1 x_3 + 2mq^2 x_2 x_3 + mq^2 x_3^2$$

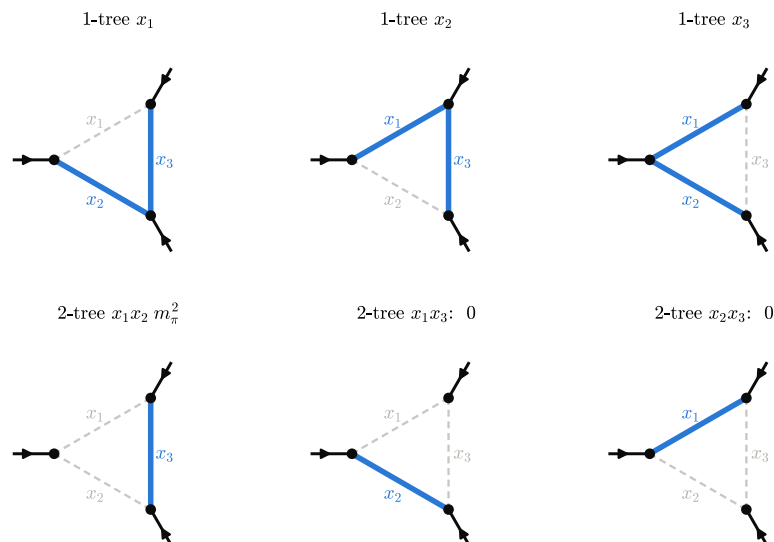
Out [42]:



Now the rules of Part 2, drawn. One loop, so a 1-tree erases one line and a 2-tree erases two. Each 2-tree cuts off one vertex. Only the one that cuts off the pion vertex P carries momentum, $(k_1 + k_2)^2 = m_\pi^2$. The two that cut off a photon vertex carry $k_1^2 = 0$ or $k_2^2 = 0$ and drop out:

```
In [43]: trees = tri.spanning_trees(); two = tri.two_forests()
lab = lambda rem: "$" + "".join("x_{%d}" % (i + 1) for i in rem) + "$"
def term(rem, comp):
    P2 = tri.kin.square_external(momentum_in(tri, comp))
    return lab(rem) + (";\m_\pi^2$" if P2 != 0 else ": 0")
fig = draw_panels(tri, trees + [r for r, c in two],
                  titles=["1-tree " + lab(r) for r in trees]
                      + ["2-tree " + term(r, c) for r, c in two],
                  momenta=False, ncols=3, size=2.0)
```

Out [43]:



So $\mathcal{U} = x_1 + x_2 + x_3$ and $V = x_1 x_2 m_\pi^2$. With the mass term $\mathcal{F} = -V + \mathcal{U}(x_1 + x_2 + x_3)m^2$ (Minkowski sign), which is the $\mathcal{F}$ printed above. On the simplex $x_1 + x_2 + x_3 = 1$, with $x_1 = x$ and $x_2 = y$, the polynomials become $\mathcal{U} = 1$ and

```
In [44]: x, y = var('x y')
m, m_pi = var('m m_pi')
Delta = SR(str(tri.F())).subs(x1=x, x2=y, x3=1 - x - y).expand() # U = 1 on the simplex
Delta = Delta.subs({SR.var('mq'): m, SR.var('mpi2'): m_pi^2})
Delta
```

```
Out[44]: -m_pi^2*x*y + m^2
```

This is the Δ of the textbook calculation, read off the trees without completing any square.

9.2 Step 2: the integral from $\mathcal{U}$ and $\mathcal{F}$

Now put $\mathcal{U}$ and $\mathcal{F}$ into the general formula of Part 2. Here $N = 3$ lines, all with power 1, $L = 1$ loop and $D = 4$ (the triangle is finite):

$$\int \frac{d^4\ell}{i\pi^2} \frac{1}{D_1 D_2 D_3} = (-1)^3 \Gamma(3-2) \int_0^\infty dx_1 dx_2 dx_3 \delta(1-x_1-x_2-x_3) \frac{\mathcal{U}^{3-4}}{\mathcal{F}^{3-2}} = - \int d^3x \delta(\dots) \frac{1}{\mathcal{U}\mathcal{F}}$$

feynsage writes the integrand straight from the polynomials of Step 1:

```
In [45]: # the general formula with N = 3 lines (powers 1), L = 1 loop, D = 4, Minkowski:
# Int d^4l/(i pi^2) 1/(D1 D2 D3)
# = (-1)^N Gamma(N - L D/2) Int dx delta(1 - sum x) U^(N-(L+1)D/2) / F^(N-L D/2)
N, L, Dv = 3, 1, 4
print("prefactor (-1)^N Gamma(N - L D/2) =", (-1)^N*gamma(N - L*Dv/2),
      " power of U:", N - (L + 1)*Dv/2, " power of F:", -(N - L*Dv/2))
Ux, Fx = SR(str(tri.U())), SR(str(tri.F()))
integrand = (-1)^N*gamma(N - L*Dv/2) * Ux^(N - (L + 1)*Dv/2) / Fx^(N - L*Dv/2)
integrand = integrand.subs(x3=1 - x1 - x2)
integrand = integrand.subs({SR.var('mq'): m, SR.var('mpi2'): m_pi^2}).simplify_full()
print("integrand on the simplex x1 + x2 + x3 = 1:", integrand)

prefactor (-1)^N Gamma(N - L D/2) = -1 power of U: -1 power of F: -1
integrand on the simplex x1 + x2 + x3 = 1: 1/(m_pi^2*x1*x2 - m^2)
```

That is $-1/\Delta$ on the simplex, so the triangle is $-\int_0^1 dx \int_0^{1-x} dy 1/(m^2 - xym_\pi^2) = -I(r)/m^2$ with

$$I(r) = \int_0^1 dx \int_0^{1-x} dy \frac{1}{1 - xyr}, \quad r = \frac{m_\pi^2}{m^2}$$

Do the integral numerically from $\mathcal{U}$ and $\mathcal{F}$ (here $m = 1$, $r = 0.5$) and compare with the closed form $I(r) = (2/r) \arcsin^2(\sqrt{r}/2)$:

```
In [46]: from scipy.integrate import dblquad
mv, rv = 1.0, 0.5 # m = 1 and m_pi^2 = r m^2 with r = 0.5
f_num = fast_callable(integrand.subs(m=mv, m_pi=sqrt(rv)),
                      vars=[SR.var('x1'), SR.var('x2')])
val = dblquad(lambda b, a: f_num(a, b), 0, 1, 0, lambda a: 1 - a)[0]
print("Int d^4l/(i pi^2) 1/(D1 D2 D3) from U and F: ", val)
print("-I(r)/m^2 with I(r) = (2/r) arcsin^2(sqrt(r)/2):",
      -(2/rv*arcsin(sqrt(rv)/2)^2)/mv^2)

Int d^4l/(i pi^2) 1/(D1 D2 D3) from U and F: -0.5223447929624236
-I(r)/m^2 with I(r) = (2/r) arcsin^2(sqrt(r)/2): -0.522344792962424
```

The amplitude uses the measure $d^4\ell/(2\pi)^4$, which is $i\pi^2/(2\pi)^4$ times the measure of the formula:

```
In [47]: Ir = var('I_r')
# the loop measure of the amplitude is d^4l/(2 pi)^4 = (i pi^2/(2 pi)^4) d^4l/(i pi^2)
(I*pi^2/(2*pi)^4 * (-Ir/m^2)).simplify_full()
```

```
Out[47]: -\frac{i I_r}{16 \pi^2 m^2}
```

For $I(r)$ itself, expand $1/(1 - xyr)$ in powers of r , integrate term by term and compare with the closed form.

```
In [48]: r = var('r')
term_k = lambda k: integrate(integrate((x*y)^k, y, 0, 1 - x), x, 0, 1)
series_I = sum(r^k * term_k(k) for k in range(10))
closed = 2/r * arcsin(sqrt(r)/2)^2
print("I(r) term by term: ", series_I.series(r, 4).truncate())
print("closed form, series:", closed.taylor(r, 0, 3))
print("heavy quark, r -> 0:", limit(closed, r=0))
print("m = 330 MeV: 2 I(r) =", (2*closed.subs(r=(134.9768/330)^2)).n(digits=5))
```

```
I(r) term by term: 1/1120*r^3 + 1/180*r^2 + 1/24*r + 1/2
closed form, series: 1/1120*r^3 + 1/180*r^2 + 1/24*r + 1/2
heavy quark, r -> 0: 1/2
m = 330 MeV: 2 I(r) = 1.0143
```

For a heavy quark $I \rightarrow 1/2$, the area of the triangle. Even for a light constituent quark ($m = 330$ MeV) $2I(r)$ is only 1.4% above 1.

9.3 Step 3: the Dirac trace

The numerator of the diagram is a trace of Dirac matrices. The pion vertex gives γ_5 , each photon vertex a γ^μ or γ^ν and each quark propagator its numerator $\gamma \cdot p + m$, where $\gamma \cdot p = \gamma^\mu p_\mu$ (often written with a slash through p). With the momenta of the triangle and the loop momentum shifted to q ($\ell = q - xk_1 + yk_2$, as in Step 2) the trace is

$$\text{Tr}[\gamma_5 (\gamma \cdot (q - xk_1 - (1-y)k_2) + m) \gamma^\nu (\gamma \cdot (q - xk_1 + yk_2) + m) \gamma^\mu (\gamma \cdot (q + (1-x)k_1 + yk_2) + m)]$$

FEYNSAGE FUNCTION `form.dirac_trace(factors, vectors, dim=4, levi_civita="form")`

The trace of a product of Dirac matrices, done by the program FORM. The matrices are given as a list, in the order of the product. Each entry is one matrix:

'g5'	γ_5
'mu' (any name that is not a momentum)	γ^μ
'p' (a momentum)	$\gamma \cdot p = \gamma^\mu p_\mu$
'p + m'	$\gamma \cdot p + m$ (m times the unit matrix)
'q - x*k1 + y*k2 + m'	$\gamma \cdot (q - xk_1 + yk_2) + m$

- **vectors:** the names that are momenta (so that 'q' means $\gamma \cdot q$ and not an index)
- **dim=4:** four dimensions, needed with γ_5 . A symbol such as 'D' gives D dimensions
- **levi_civita="usual":** write the answer with the usual Levi-Civita symbol $\varepsilon^{\mu\nu\rho\sigma}$ (Peskin and Schroeder, $\varepsilon^{0123} = +1$, so $\text{Tr}[\gamma^\mu \gamma^\nu \gamma^\rho \gamma^\sigma \gamma_5] = -4i\varepsilon^{\mu\nu\rho\sigma}$, their eq. (5.5)), printed ε . The default "form" keeps FORM's own symbol, which is $-i$ times it.
- returns a Sage expression. `g(mu,nu)` is $g^{\mu\nu}$ and `dot(p,q)` is $p \cdot q$. In $\varepsilon(\dots)$ a momentum in a slot means it is contracted there, so $\varepsilon(k_1, k_2, \mu, \nu) = \varepsilon^{\rho\sigma\mu\nu} k_{1\rho} k_{2\sigma}$.

```
In [49]: # Tr[ g5 (q - x k1 - (1-y) k2 + m) g^nu (q - x k1 + y k2 + m) g^mu
#         (q + (1-x) k1 + y k2 + m) ]
# each momentum combination stands for gamma.(that momentum);
# the answer with the usual epsilon
form.dirac_trace(['g5', 'q - x*k1 - (1-y)*k2 + m', 'nu', 'q - x*k1 + y*k2 + m', 'mu',
                  'q + (1-x)*k1 + y*k2 + m'],
                 vectors=['q', 'k1', 'k2'], levi_civita="usual")
```

```
Out [49]: 4im\varepsilon(k1, k2, \mu, \nu)
```

So the whole numerator is one term,

$$4im \varepsilon(k_1, k_2, \mu, \nu) = 4im \varepsilon^{\mu\nu\rho\sigma} k_{1\rho} k_{2\sigma}$$

(moving the two momentum slots past the two index slots is an even permutation). Nothing depends on q , x or y , so the loop integral of Step 2 is all that is needed. It is a pure scalar triangle. Only the terms with exactly one m survive, because a trace with γ_5 needs an even number, at least four, of other γ matrices.

9.4 Step 4: the amplitude and why the quark mass drops out

Collect the factors of one quark. They are N_c colours, $(eQ)^2$ from the photon vertices, g from the pion vertex, $4m$ from the trace (Step 3), $I/(16\pi^2 m^2)$ from the integral (Step 2) and a factor 2 for the two orders of the photons. Then put in $g = m/f_\pi$ and add u and d (with $\tau_3 = \pm 1$):

```
In [50]: Nc, e, Q, g, f, mq_ = var('N_c e Q g f_pi m_q')
# one quark, both orderings of the photons, heavy quark
A_one = Nc * e^2 * Q^2 * g / (4*pi^2*mq_) * 2*limit(closed, r=0)
print("one quark:", A_one)
print("with g = m_q/f_pi:", A_one.subs(g=mq_/f))
alpha = var('alpha')
A = A_one.subs(g=mq_/f, Q=2/3) - A_one.subs(g=mq_/f, Q=-1/3) # u minus d
A = A.subs(e=sqrt(4*pi*alpha))
print("u and d, e^2 = 4 pi alpha:", A.simplify_full())

one quark: 1/4*N_c*Q^2*e^2*g/(pi^2*m_q)
with g = m_q/f_pi: 1/4*N_c*Q^2*e^2/(pi^2*f_pi)
u and d, e^2 = 4 pi alpha: 1/3*N_c*alpha/(pi*f_pi)
```

The m of the coupling cancels the $1/m$ of the loop. A heavier quark couples more strongly and propagates less. The answer knows only the charges and the number of colours, $A = N_c(Q_u^2 - Q_d^2)e^2/(4\pi^2 f_\pi) = \alpha/(\pi f_\pi)$ for $N_c = 3$. This is the axial anomaly of Adler, Bell and Jackiw.

9.5 Step 5: the decay rate

Square the amplitude and sum over the photon polarisations. That needs the contraction $\varepsilon^{\mu\nu\rho\sigma}\varepsilon_{\mu\nu\alpha\beta}k_{1\rho}k_{2\sigma}k_1^\alpha k_2^\beta$. It is written exactly as on paper in Sage notation. `form.compute` turns it into a FORM program, runs it and reads the answer back. No FORM code has to be written (`show_code=True` prints the program it made):

FEYNSAGE FUNCTION `form.compute(expr, vectors, lines=(), rules=None, dim=4, show_code=False)`

Traces and index contractions, written in Sage notation and done by FORM.

- `expr`: a product such as `eps(mu,nu,rho,sigma)*k1(rho)`, with `eps(...)` the Levi-Civita symbol, `g(mu,nu)` the metric, `dot(p,q)` or `p.q` a scalar product, `p(mu)` the component p^μ and any other symbols
- `lines`: fermion lines to trace, each a list of factors as in `dirac_trace`. For example `[["pp","mu","p","nu"], ["k","mu","kp","nu"]]` is the product of two traces of $e^+e^- \rightarrow \mu^+\mu^-$
- `rules`: scalar products to put in, for example `{"k1.k1": 0, "k1.k2": "s/2"}`
- `dim`: 4, or a symbol like 'D' for traces in D dimensions
- `levi_civita="usual"`: `eps` is the usual $\varepsilon^{\mu\nu\rho\sigma}$, in the input and in the answer (otherwise FORM's own symbol, $-i\varepsilon$)
- every repeated index is summed. Returns a Sage expression, with `dot(p,q)` for $p \cdot q$.

```
In [51]: # Sage-style input; feynsage writes the FORM program, runs it and reads the answer back
form.compute("eps(mu,nu,rho,sigma)*eps(mu,nu,al,be)*k1(rho)*k2(sigma)*k1(al)*k2(be)",
            vectors=["k1", "k2"], rules={"k1.k1": 0, "k2.k2": 0},
            levi_civita="usual", show_code=True)
```

```
Vectors k1,k2;
Indices al,be,mu,nu,rho,sigma;
Local E = e_(mu,nu,rho,sigma)*e_(mu,nu,al,be)*k1(rho)*k2(sigma)*k1(al)*k2(be);
contract;
id k1.k1 = 0;
id k2.k2 = 0;
Format nospaces;
Print +s;
.end
```

```
Out [51]: 2 dot(k1, k2)^2
```

The printed program shows what was done for us. It has FORM's `e_` for ε (with the factor i taken care of), the indices declared, `contract` and the two rules $k_1^2 = k_2^2 = 0$. The answer is $\varepsilon^{\mu\nu\rho\sigma}\varepsilon_{\mu\nu\alpha\beta}k_{1\rho}k_{2\sigma}k_1^\alpha k_2^\beta = 2(k_1 \cdot k_2)^2 = m_\pi^4/2$.

WRITING FORM YOURSELF

For anything `compute` does not cover, `form.run_form(code)` runs a FORM program written by hand and returns FORM's output. A pedestrian introduction to FORM for particle physics (declarations, `g_` and `trace4`, `e_` and `contract`, `id` and a full Compton-scattering calculation) is on my blog, rousan.netlify.app/pages/physics/blogs/form_pedestrian_qft_blog.

With the two-body phase space $1/(8\pi)$, a factor $1/2$ for the two identical photons and $1/(2m_\pi)$ for the decaying pion,

$$\Gamma = \frac{1}{2m_\pi} \cdot \frac{1}{2} \cdot \frac{1}{8\pi} \cdot A^2 \frac{m_\pi^4}{2} = \frac{A^2 m_\pi^3}{64\pi}$$

Now the numbers, with $\alpha = 1/137.036$, $m_{\pi^0} = 134.977$ MeV and $f_\pi = 130.2/\sqrt{2}$ MeV (Particle Data Group), against the measurements. The lifetime follows from the width as $\tau = \hbar \text{BR}(\gamma\gamma)/\Gamma$:

```
In [52]: mpi = var('m_pi')
Gamma = (A^2 * mpi^3 / (64*pi)).simplify_full()
print("Gamma =", Gamma)
vals = {alpha: 1/137.035999, mpi: 134.9768, f: 130.2/sqrt(2)} # MeV (PDG values)
hbar, BR = 6.582119569e-22, 0.98823 # MeV s, BR(pi0 -> gamma gamma)
G3 = (Gamma.subs(vals).subs({Nc: 3}) * 1e6).n() # eV
G1 = (Gamma.subs(vals).subs({Nc: 1}) * 1e6).n()
tau = lambda G: hbar*1e6*BR/G # s, G in eV
sig = lambda a, b, err: abs(a - b)/err # distance in standard deviations
print("width feynsage %.2f eV" % G3)
print("lifetime feynsage %.3g s PDG world average (8.43 +- 0.13)e-17 s %.1f sigma"
      % (tau(G3), sig(tau(G3), 8.43e-17, 0.13e-17)))
print("with N_c = 1: %.2f eV, nine times too small" % G1)
vals_f = dict(vals); vals_f[f] = 1.01*130.2/sqrt(2)
Gf = (Gamma.subs(vals_f).subs({Nc: 3})*1e6).n()
print("f_pi 1% larger: Gamma = %.2f eV, 2% lower (Gamma ~ 1/f_pi^2)" % Gf)

Gamma = 1/576*N_c^2*alpha^2*m_pi^3/(pi^3*f_pi^2)
width feynsage 7.79 eV
lifetime feynsage 8.35e-17 s PDG world average (8.43 +- 0.13)e-17 s 0.6 sigma
with N_c = 1: 0.87 eV, nine times too small
f_pi 1% larger: Gamma = 7.63 eV, 2% lower (Gamma ~ 1/f_pi^2)
```

RESULT: THE PION LIFETIME FROM ONE TRIANGLE

	feynsage	experiment
width $\Gamma(\pi^0 \rightarrow \gamma\gamma)$	7.79 eV	
lifetime, world average	8.35×10^{-17} s	$(8.43 \pm 0.13) \times 10^{-17}$ s (PDG): 0.6σ
the same with $N_c = 1$	0.87 eV	ruled out

READING THE COMPARISON

- Our lifetime is 1% shorter than the world average of the Particle Data Group, which is 0.6 of its standard deviation. There is no tension.
- The prediction has its own small uncertainty. Since $\Gamma \propto 1/f_\pi^2$, so 1% in f_π moves it by 2%. The corrections left out (the masses of the u and d quarks, mixing with η and η') are of a few per cent as well.

- With one colour the rate would be nine times smaller and the pion would live nine times longer, against every measurement. This decay was one of the early pieces of evidence that quarks come in three colours. The quark mass does not matter (Step 4), which is the content of the anomaly.

10 Where to go next

WHAT FEYNSAGE IS FOR

feynsage is meant for small and medium problems and for studying. Every step is exact, can be checked and explains itself. For very large reductions Kira and FIRE remain the better choice. I may continue improving feynsage in the future.

- github.com/aburousan/feynsage: the code and the latest version.
- [examples/feynsage_walkthrough.ipynb](#): a notebook that goes through the whole package.
- My notes of the lecture, *Feynman integrals at one loop*, with every derivation in full. They are not finished yet and will be shared once complete.
- References used in the lecture
 - P. Ramond, *Field Theory: A Modern Primer* (2nd ed.), Sections 4.3 and 4.4 and Appendix B
 - M. Argeri and P. Mastrolia, *Feynman diagrams and differential equations*, Int. J. Mod. Phys. A 22 (2007) 4375 (arXiv:0707.4037)
 - V. A. Smirnov, *Feynman Integral Calculus* (Springer 2006), chapter 3
 - S. Weinzierl, *Feynman Integrals* (Springer 2022, arXiv:2201.03593), Section 2.5 and Chapter 3
 - T. Rauh, *Introduction to Feynman integrals and multiloop techniques* (lecture notes, 2019). They follow Smirnov's later book, *Analytic Tools for Feynman Integrals* (Springer 2012)
- For the pion
 - S. L. Adler (1969) and J. S. Bell and R. Jackiw (1969)
 - Particle Data Group, Review of Particle Physics